SIB Doc: Secure NL-to-SPARQL over Clinical Knowledge Graphs

· SIB / SPHN challenge · Neural Wave hackathon 2026 ·

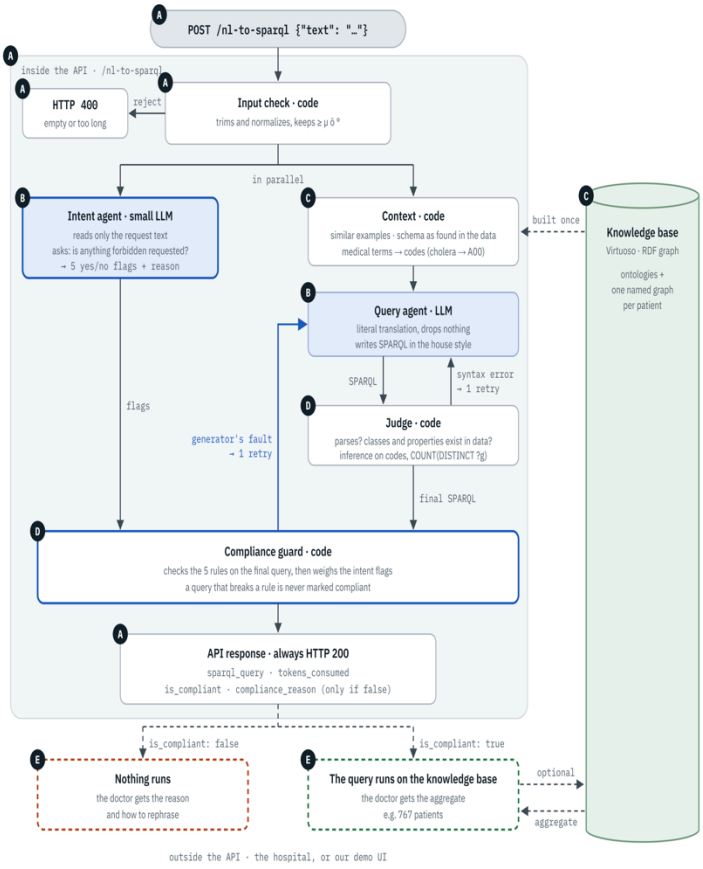
Team Epsilon Dev

Santiago Figueroa Luchetti · Adrika Pradhan · Gayathri Girish · David García Costa · Odinaka Ogbujiagba

1. Goal and Challenge

SIB Doc translates natural-language research questions into SPARQL over SPHN clinical knowledge graphs and checks five privacy rules: no identifiable-patient selection, individual-level results, unrestricted scans, non-SELECT queries or identifier disclosure. Each patient occupies one named graph, so patient counts use COUNT(DISTINCT ?g). The open-source system returns queries, verdicts and total token usage through a fixed REST API without accessing a database. Evaluation uses organizer-provided synthetic data modeled with SPHN schema 2026.1.

2. Approach and Model Architecture



- ➔ **Input check** normalizes Unicode and whitespace, removes control characters and emoji, and rejects empty or oversized text with HTTP 400. Clinical symbols and accented characters are supported.
- ➔ **Intent agent (LLM)** reads the original request and returns five privacy flags, a reason and a possible rephrasing. It runs in parallel with query generation to reduce latency.
- ➔ **Context Retrieval (code)** supplies up to three similar BM25 examples, eight schema cards describing observed classes and properties, and five terminology candidates. The evaluated configuration used a local index of 38,765 codes. Context is prepared offline; the judged request path never accesses a database.
- ➔ **Query agent (LLM)** translates the complete request into SPARQL using the retrieved context. It preserves forbidden parts so an unsafe request cannot become compliant through silent omission. Both agents use self-hosted Qwen3.6-27B in 4-bit AWQ, with v4 prompts, temperature 0 and thinking disabled.
- ➔ **Judge (code)** repairs unambiguous syntax, scope and aggregation errors, then checks schema terms, class/property compatibility, variable scope and question coverage. Remaining errors may trigger bounded retries with specific correction hints.
- ➔ **Compliance guard (code)** analyzes the SPARQL algebra against the five privacy rules. It tracks sensitive values through expressions, bindings, subqueries and aggregates, then combines its findings with the intent flags. Flagged requests are never retried, and R1 violations cannot trigger guard retries.
- ➔ The API returns the query, compliance verdict, reason when non-compliant, and total model token usage without executing SPARQL. The researcher console presents released aggregates as tables and charts, supports exports and explicitly confirmed sharing, and optionally executes compliant queries against a development database. This execution path is separate from the judged API.

3. Evaluation and Test Results

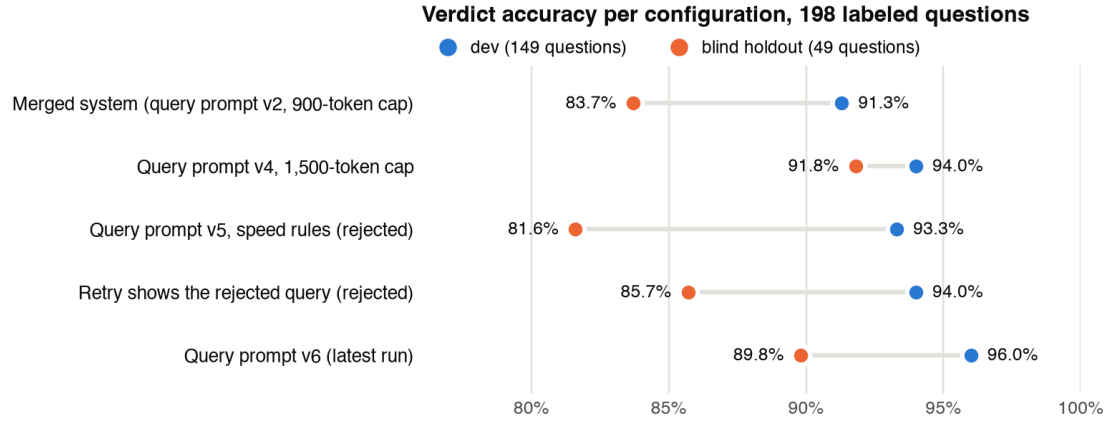
We evaluated privacy verdicts on **198 labeled requests**, split into **149 development** and **49 holdout** questions. These cover official examples, paraphrases, look-alikes, mixed requests, prompt injection, gray zones, shorthand and multilingual inputs. Holdout questions were excluded from case-by-case tuning, although aggregate scores informed configuration selection.

Query correctness was evaluated separately on **87 questions with verified reference SPARQL**: 11 permitted official examples, 45 development questions and 31 holdout questions. Development references also serve as retrieval examples, making their scores less informative about generalization.

The final system achieved:

- **99.3% development and 100% holdout verdict accuracy**, with zero forbidden requests marked compliant.
- **53/87 answers matching the reference's numbers**, including **34/87 exact results** and **20/31 matching-number answers on the query holdout**. Matching numbers allows differences in labels, layout or small rounding.
- **7.2 seconds median and 20.1 seconds p95** for translation and checking at concurrency four, excluding database execution.
- **1,696 passing local Python tests**, with one existing third-party deprecation warning.

An earlier 27B run timed **80 generated queries** against references on the **100,000-patient synthetic database**. Both sides completed within the 30-second deadline for **58 pairs**. Among those completed pairs, median execution times were **0.3 seconds for generated queries** and **0.5 seconds for references**. These exploratory measurements predate the final fixes and do not reproduce the jury's ten-isolated-run protocol.



Qwen3.5-9B, intent prompt v4, 17 October 2026, 4 requests at a time. A repeat of the same configuration differed by about 1–2 points.

4. Metrics

Metric	Latest recorded result
Compliance verdict accuracy	**99.3%** development (148/149); 100% holdout (49/49)
Forbidden requests marked compliant	**0/117** across both splits
Exact query results	**34/87**
Answers matching the reference's numbers[1]	**53/87,** including 20/31 query-holdout questions
Generated queries executable on development Virtuoso	**92%**

Queries passing the judge on the first attempt	**85%**
Translation and checking latency[2]	**7.2 s** median; 20.1 s p95
Local Python regression tests	**1,696 passed**, with one existing deprecation warning

Privacy verdicts were evaluated on 198 labeled requests: 149 development and 49 holdout. Query correctness used 87 verified references: 11 official, 45 development and 31 holdout, executed on the 2,000-patient synthetic database. Development references also support retrieval; holdout questions were excluded from case-by-case tuning, although aggregate scores informed configuration selection.

5. Challenges and Tradeoffs

- **Privacy and answer correctness require separate checks.** A compliant query can answer the wrong question. The system achieved 100% verdict-holdout accuracy, while matching the reference's numbers on 20/31 query-holdout questions.
- **Observed data differs from the nominal schema.** Valid-looking properties can return no matches, and versioned clinical codes can split equivalent concepts. Offline schema cards and targeted repairs address these issues, but the snapshots need rebuilding for another dataset.
- **Retries trade latency for possible improvements.** Regeneration can correct errors or introduce new ones. Deterministic repairs handle unambiguous mistakes first; remaining retries are bounded. Flagged requests are never regenerated because a retry could silently remove their forbidden parts.
- **Model quality comes at a throughput cost.** Qwen3.6-27B improved query answers and reduced retries compared with the 9B model. However, measured throughput under eight concurrent clients fell from 1.33 to 0.65 requests per second.
- **More context does not guarantee better answers.** Three retrieved examples outperformed two, but adding 41 verified examples reduced query-holdout performance. Restricted terminology assets improve grounding but cannot accompany the public repository, limiting reproduction of the measured results.
- **Static privacy checks provide bounded protection.** They check the five challenge rules without database access, but cannot prove clinical correctness or prevent inference through repeated aggregate requests. Optional small-group suppression provides a separate release policy and is disabled by default.

6. Ideas that did not work out

- **Code-specialized models.** Early tests of larger coder models did not improve exact query results, although they answered faster; one also flagged more allowed requests as forbidden. These alternatives were rejected. Later, Qwen3.6-27B improved query correctness after retrieval and deterministic checks had been strengthened, so we adopted it.
- **Showing the model its own mistake.** Giving the model its rejected query encouraged it to copy the broken structure. Valid judge retries fell from **8/18 to 3/19**, and executable queries fell from **67% to 48%**. Retry feedback now provides specific errors and correction hints without the rejected query.
- **Giving forbidden requests another chance.** A retry could silently drop a patient-specific restriction, producing a safe-looking query for an unsafe request. Requests flagged by the intent agent are now never retried, and an R1 violation cannot trigger a guard retry.

7. Use of AI agents and coding assistants

The evaluated system uses self-hosted **Qwen3.6-27B in 4-bit AWQ** for both intent classification and SPARQL generation. No proprietary model is called by the running translation and compliance pipeline.

During development, we used Claude Code and Codex to assist with implementation, code review, tests and documentation. AI assistance also supported the architecture diagram, accuracy chart and report drafting. These assistants are development tools, separate from the models used by the shipped system.

AI-assisted code was checked through automated regression tests, while model and prompt choices were assessed using privacy-verdict and reference-query evaluations. Report claims were cross-checked against repository evidence.

SIB/SPHN supplied the challenge, schema, mocked dataset, examples and database setup. Project code is MIT licensed; restricted patient and terminology material is excluded from redistribution.